

7. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: Friday, July 17, 2026 | **Scope:** `target/` — Java 8 / Spring Boot 2.7.18 / JPA / JSP / AngularJS 1.8 (`glmarvel29-ai/global-hr-modernization` on GitHub `main`) plus local React 18 apps (`social-media-react` , `workbench-demo`) and AI workbench scaffolding (`.cursor/` , `.kiro/`)

Executive Summary

EXECUTIVE SUMMARY

The `TARGET_WORKSPACE` combines a compact **ERM Complexity Demo** Spring Boot monolith (31 files on GitHub `main`, zero CI, zero tests) with two **React 18 frontends** used as workbench targets. Repository hygiene is uneven: there is no top-level `.gitignore`, `workbench-demo` lacks any `.gitignore`, and `target/.env` plus `.cursor/mcp.json` hold integration tokens locally without workspace-level exclusion rules. No `.github/workflows`, no `.env.example`, and no `container/devcontainer` path exist anywhere in scope, so onboarding and agent-authored changes cannot be verified automatically. Database posture relies on `spring.jpa.hibernate.ddl-auto=update` with a single flat `risk_register` table and plaintext credentials in profile properties. AI tooling (`.cursor/`, `.kiro/`, `.claude/`, MCP bundles) is present but immature — there is no CI gate to trust agent output. **Overall agentic-harness readiness is High Risk**, driven by absent CI, secrets-adjacent repo hygiene, and non-reproducible dev setup.

Partial

TOP-LEVEL .GITIGNORE
PRESENT

0

CI/CD WORKFLOWS FOUND

13 / 15

THIRD-PARTY PACKAGES
DECLARED / WIRED

No

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

High Risk

Driven by High Risk gaps in code repository health (D1), database usage (D4), development environment (D5), and committed-adjacent secrets hygiene (D6) — no CI gate exists to validate agent-authored changes.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	-----------	------	----------	-----------	----------	--------

D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	No CI (0 workflows); no top-level <code>.gitignore</code> ; <code>workbench-demo</code> missing <code>.gitignore</code> ; lock files present for npm apps; Maven <code>pom.xml</code> committed on GitHub	HIGH RISK
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused or unmaintained	13 of 15 infra-relevant packages wired; 2 unused npm deps (<code>dotenv</code> , <code>jwt-decode</code>); Spring Boot 2.7.18 EOL-adjacent	MODERATE
D3	AI Tool / Agentic Readiness	ready	partial	not ready	<code>.cursor/</code> , <code>.kiro/</code> , <code>.claude/</code> + MCP configs present; layered Java modules and React feature folders are enumerable; no CI gate or uniform patterns across subprojects	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	<code>ddl-auto=update</code> ; no Flyway/Liquibase; single flat <code>risk_register</code> table; no FK/index beyond PK; idempotent seed loader; plaintext DB passwords in profile properties	HIGH RISK
D5	Development Environment	reproducible	partial	manual / fragile	No <code>.env.example</code> , Dockerfile, or devcontainer; README runs <code>mvn package -DskipTests</code> ; ESLint configured but not enforced in CI; <code>324M node_modules</code> in <code>workbench-demo</code> with no local <code>.gitignore</code>	HIGH RISK
D6	Secrets & Config Hygiene (additional)	no secrets in tree	isolated local-only	tokens in workspace files	<code>target/.env</code> and <code>target/.cursor/mcp.json</code> contain live integration tokens; no root <code>.gitignore</code> to exclude them	HIGH RISK
D7	Documentation Integrity (additional)	docs match repo	minor drift	broken references	<code>README.md</code> links to <code>docs/START_TEST_AND_ISSUES.md</code> and <code>docs/COMPLEXITY_ISSUES.md</code> but <code>docs/</code> returns 404 on GitHub <code>main</code>	MODERATE

7.1 Code Repository Health

Check	Files Inspected	Finding	Consequence	Next Step
Top-level <code>.gitignore</code>	<code>target/</code> root (absent); <code>global-hr-modernization/.gitignore:1-8</code> (GitHub)	No top-level <code>.gitignore</code> under <code>target/</code> . GitHub repo has minimal Java-centric ignore list (<code>target/</code> , <code>.idea/</code> ,	Local-only secrets (<code>target/.env</code>), <code>node_modules/</code> (324M in <code>workbench-demo</code>), and AI config dirs can be accidentally staged	Add <code>target/.gitignore</code> covering <code>.env</code> , <code>**/node_modules/</code> , <code>.cursor/</code> , <code>.kiro/</code> , <code>.claude/</code> , and <code>agent-runs/</code> ; extend GitHub <code>.gitignore</code> similarly.

		<code>*.iml</code> , <code>*.log</code> , <code>.DS_Store</code>) — no coverage for <code>.env</code> , <code>node_modules/</code> , or MCP config dirs.	in a combined workbench checkout.	
Per-app <code>.gitignore</code>	<code>target/social-media-react/.gitignore:1-25</code> ; <code>target/workbench-demo/</code> (none)	<code>social-media-react</code> ignores <code>/node_modules</code> , <code>/build</code> , and <code>.env*</code> . <code>workbench-demo</code> has no <code>.gitignore</code> , despite committed-style layout and a local <code>.env</code> .	Any <code>git add .</code> inside <code>workbench-demo</code> risks committing dependencies and secrets.	Copy CRA-style <code>.gitignore</code> into <code>target/workbench-demo/.gitignore</code> immediately.
CI/CD presence	GitHub API <code>GET .../contents/.github</code> → 404; <code>find target -path '*/.github/workflows/*'</code> (only hits inside <code>node_modules</code>)	Zero project-owned CI workflows across the ERM demo and both React apps.	Lint, test, and build are never enforced before merge; agent-authored patches cannot be auto-validated.	Add <code>.github/workflows/ci.yml</code> on <code>global-hr-modernization</code> running <code>mvn verify</code> ; add parallel npm CI for each React app.
Branch protection signals	Repo tree on GitHub <code>main</code> (31 blobs); <code>target/</code> scan for <code>CODEOWNERS</code> , <code>pull_request_template.md</code>	No <code>CODEOWNERS</code> , PR template, or required-checks config visible locally or on GitHub.	Unreviewed changes can land on <code>main</code> without automated or ownership gates.	Add <code>.github/CODEOWNERS</code> and a PR template referencing CI checks once workflows exist.
Lock files	<code>target/social-media-react/package-lock.json</code> ; <code>target/workbench-demo/package-lock.json</code> ; <code>global-hr-modernization/pom.xml</code> (GitHub)	npm lock files committed for both React apps; Maven coordinates pinned via <code>spring-boot-starter-parent 2.7.18</code> in <code>pom.xml</code> .	npm installs are reproducible; Java deps inherit BOM versions but no <code>spring-boot-starter-test</code> is declared.	Add <code>spring-boot-starter-test</code> to <code>pom.xml</code> ; keep lock files updated via Dependabot once CI exists.

No files matched `target/workbench-demo/**` .

7.2 Third-Party Tools Usage

Package	Declared	Actually Wired?	Debt
<code>spring-boot-starter-web</code>	Y (<code>pom.xml</code>)	Y (<code>ApiController</code> , <code>PageController</code>)	On Spring Boot 2.7.18 (maintenance mode); plan upgrade path.
<code>spring-boot-starter-data-jpa</code>	Y	Y (<code>RiskItemRepository</code> , <code>RiskService</code>)	Schema managed via <code>ddl-auto=update</code> , not migrations.

h2	Y	Y (application.properties:16-21)	Default in-memory demo DB; H2 console enabled (/h2-console).
postgresql	Y	Y (application-postgres.properties:4-8)	Plaintext erm/erm credentials in committed profile file.
ojdbc8	Y	Y (application-oracle.properties:4-8)	Plaintext credentials; Oracle profile for legacy dialect demo.
tomcat-embed-jasper + jstl	Y	Y (WebMvcConfig , JSP views under WEB-INF/jsp/)	Legacy JSP stack increases agent refactor surface.
SAP / Oracle ERP / ServiceNow SDKs	N	N (stubs only in IntegrationHub.java:17-36)	Integration names are hard-coded demo data, not real connectors — acceptable for demo but misleading if treated as wired.
axios (social-media-react)	Y (package.json:15)	Y (httpService.js:1-39)	Pinned to ^0.27.2 — unmaintained line with known CVEs (see security report).
axios (workbench-demo)	Y (package.json:6)	Y (authService.ts:1-9)	Wired to REACT_APP_API_BASE_URL ; pinned 1.7.2 .
socket.io-client	Y (package.json:28)	Y (socket.service.js:1)	Real-time layer depends on external API at localhost:3030 .
google-map-react	Y (package.json:17)	Y (Map.jsx:2,116-155)	API key hard-coded in component (client bundle leak).
workbox-* (6 packages)	Y (package.json:30-41)	Y (service-worker.js:10-14)	PWA scaffolding present; not validated in CI.
dotenv	Y (package.json:16)	N — no imports under src/	Dead dependency; adds audit noise. Remove from package.json .
jwt-decode	Y (package.json:18)	N — no imports under src/	Dead dependency; remove or wire into auth flow.
react-router-dom	Y (both apps)	Y	v5 in social-media-react , v6 in workbench-demo — inconsistent routing patterns for agents.

7.3 AI Tool Usage & Agentic Readiness

Existing AI-assisted tooling

Artifact	Path	Maturity
Cursor MCP config	target/.cursor/mcp.json:1-30	Present — Jira + GitHub MCP servers configured with inline env tokens
Kiro settings	target/.kiro/settings/ , target/.kiro/context/	Present — pipeline history and MCP bundles
Claude config	target/.claude/mcp.json	Present

Agent run artifacts	<code>target/agent-runs/</code> , <code>agent-runs/20260717T132536_tkr1f1/</code>	Active discovery pipeline outputs from current run
---------------------	---	--

Enumerable units of work

The ERM demo on GitHub exposes a clean package-by-layer layout (`controller/` , `service/` , `repository/` , `security/` , `integration/` , `legacy/`) with 17 Java source files — suitable for slice-by-slice agent refactors (e.g., extract `AccessControlService` tests, document `ApiController` endpoints). The React apps organize by feature (`src/pages/` , `src/cms/` , `src/services/`) but use **inconsistent state patterns** (Redux in `social-media-react` , local state in `workbench-demo`) and **two React Router major versions**, which increases agent task fragmentation.

Honest assessment: AI tooling configs exist and modules are listable, but **without CI, lock-step conventions, or a root work queue**, an agentic harness cannot safely accept bulk refactors today. Partial readiness only.

7.4 Database Usage

Check	Files Inspected	Finding	Consequence	Next Step
Schema design	<code>RiskItem.java:21-48</code> (GitHub); <code>RiskItemRepository.java:9-14</code>	Single denormalized <code>risk_register</code> entity; <code>@Id</code> only — no foreign keys, no secondary indexes beyond implicit PK.	Domain integrity (e.g., risk-to-domain cardinality) lives only in application code; extraction to microservices later requires schema rework.	Add <code>@Index</code> on <code>domain</code> and <code>severity</code> ; document intentional denormalization in a migration READ
Migration hygiene	<code>application.properties:23</code> (<code>ddl-auto=update</code>); repo tree (no <code>db/migration</code> , no Flyway/Liquibase)	No versioned migrations ; Hibernate auto-updates schema at startup.	Destructive or non-reversible schema changes cannot be reviewed, rolled back, or replayed in CI.	Introduce Flyway under <code>src/main/resources/db/migrat</code> and set <code>ddl-auto=validate</code> .
Data ownership	<code>RiskItem.java</code> + <code>DemoDataLoader.java:24-55</code>	All persistence in one shared <code>risk_register</code> table across nine risk domains — flat, not per-domain schemas.	Blocks safe bounded-context extraction; every service shares one table namespace.	Split seed data and repositories by dc package before service extraction.
Seed/sample data hygiene	<code>DemoDataLoader.java:24-27</code> (<code>if (repository.count() > 0) return;</code>)	Seeding is idempotent on empty register; uses synthetic demo	Safe for local demo; re-runs	Keep guard; add <code>@Profile("demo"</code> production profiles never auto-seed.

		narratives, not production data.	skip when data exists.	
Credentials in config	<code>application-postgres.properties:6-7</code> ; <code>application-oracle.properties:6-7</code>	Plaintext <code>username / password</code> (<code>erm/erm</code>) committed in profile files.	Credential rotation requires code changes; secrets scanners will flag repo.	Externalize to env vars (<code>SPRING_DATASOURCE_PASSWORD</code>) add <code>.env.example</code> for local profile

No files matched `**/application*.properties` containing `spring.datasource.password=`.

7.5 Development Environment

Check	Files Inspected	Finding	Consequence	Next Step
<code>.env.example</code>	<code>target/workbench-demo/.env</code> (present); <code>target/workbench-demo/.env.example</code> (absent); <code>target/social-media-react/.env*</code> (absent); GitHub repo (no <code>.env*</code>)	No <code>.env.example</code> anywhere; <code>workbench-demo</code> expects <code>REACT_APP_API_BASE_URL</code> per <code>authService.ts:6</code> but documents no variables.	New contributors and agents cannot infer required env vars; onboarding relies on tribal knowledge.	Add <code>workbench-demo/.env.example</code> with <code>REACT_APP_API_BASE_URL=https://api.example.com</code>
OS portability	<code>README.md:38-44</code> (GitHub); <code>package.json</code> scripts	Maven + JDK quick-start is cross-platform; React apps use standard CRA scripts.	Generally portable; no Windows-only tooling detected.	No action required beyond documentation already in README.
Containerization	<code>find target -maxdepth 3 \</code> <code>(-name Dockerfile -o -name docker-compose* -o -name .devcontainer.json)</code> <code>→ empty</code>	No Dockerfile, docker-compose, or devcontainer in scope.	Environment drift between contributors, CI, and agent sandboxes.	Add <code>Dockerfile</code> + <code>docker-compose.yml</code> (port 8090) and a devcontainer spec
Code style enforcement	<code>social-media-react/package.json:49-53</code> (<code>eslintConfig</code>); <code>social-media-react/README.md:56</code> (<code>npm run lint</code>); <code>social-media-react/package.json:43-47</code> (scripts — no lint script); CI search → 0 workflows	ESLint extends <code>react-app</code> but no npm lint script and no pre-commit or CI enforcement ; <code>workbench-demo</code> has no ESLint config.	Formatting and lint rules drift immediately; agents cannot rely on automated style gates.	Add <code>"lint": "eslint src --ext .js,.jsx --ignore-path .eslintignore --quiet --max-warnings 0"</code> to <code>package.json</code> and <code>npm test -- --watch</code> to <code>package.json</code>

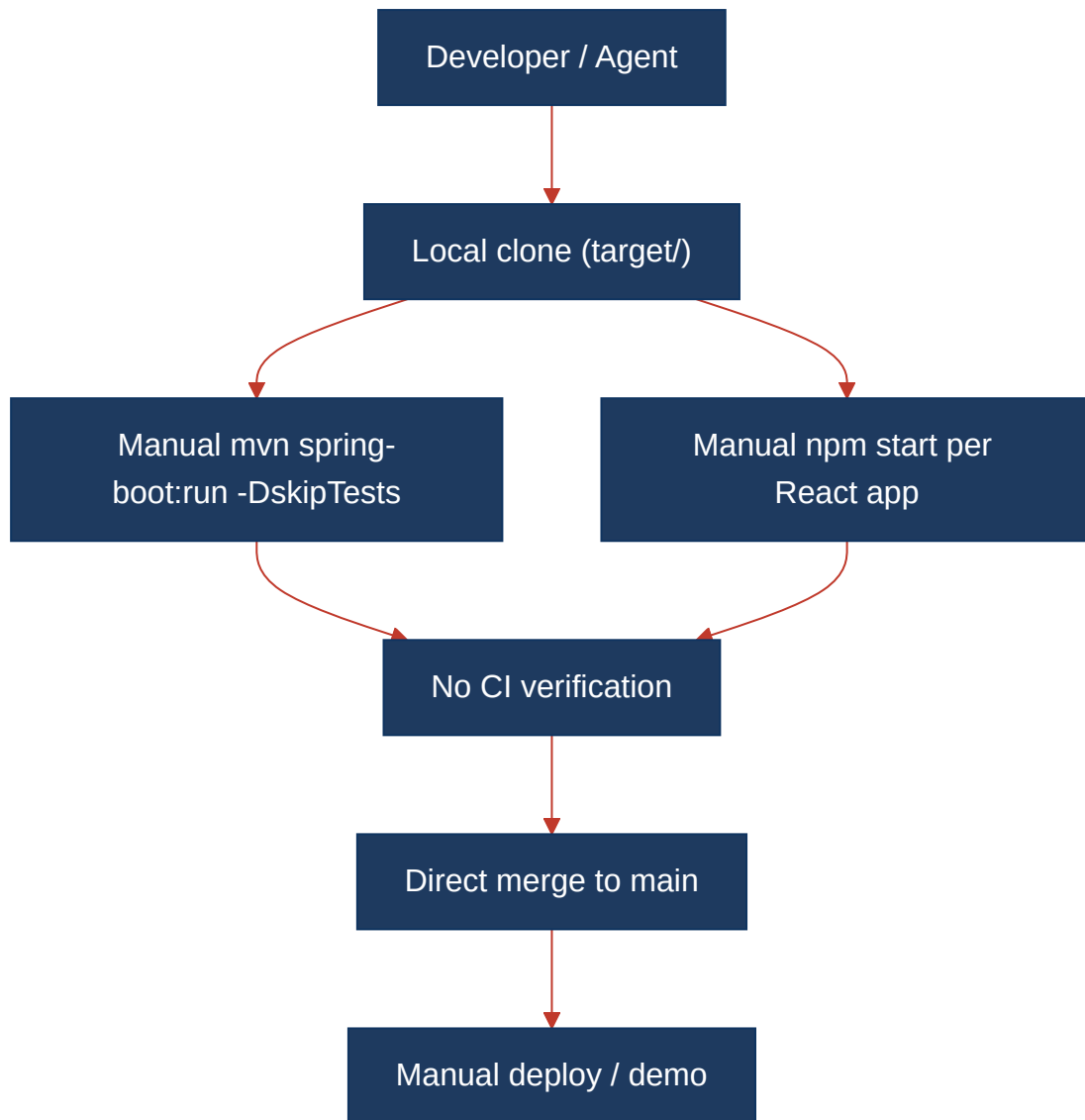
No files matched `target/social-media-react/src/**/*.js,jsx`.

7.6 Prerequisites for Agentic Harness & Marketplace Readiness

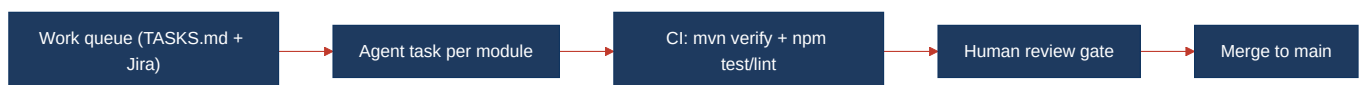
Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	Java packages by layer (17 files); React pages/components by feature (~90 files in <code>social-media-react</code>)	No central <code>TASKS.md</code> or issue-linked queue; subprojects use different stacks
Isolated, verifiable units of work	ERM controllers/services are small and injectable; React services partially centralized (<code>httpService.js</code>)	Shared Redux + socket orchestration in <code>Main.jsx</code> ; client-only auth guards; no backend in repo for React apps
CI gate to accept agent-authored output	None — 0 workflows	Agent PRs cannot be auto-verified
Repo hygiene for automation (clean checkout, no secrets)	Partial <code>.gitignore</code> on one React app; <code>target/.env</code> and MCP configs with live tokens	Secrets-adjacent files lack root exclusion; <code>workbench-demo</code> unprotected
Marketplace packaging readiness	Single Maven WAR + two standalone CRA apps; no monorepo tooling or shared versioning	No unified release manifest, changelog, or artifact publishing pipeline

7.7 Diagrams

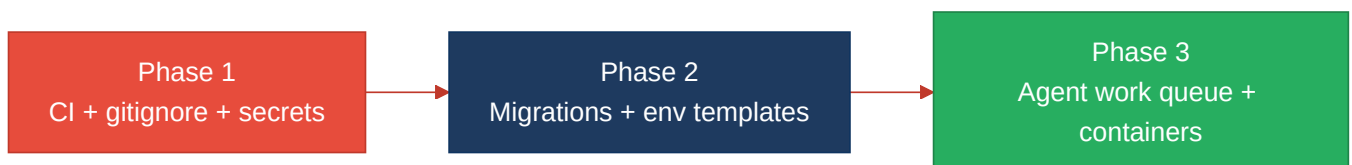
Current dev / delivery flow



Agentic harness readiness target



Improvement roadmap



7.8 Actions Required

Gap	Action	Rating	Priority
No CI/CD workflows	Add <code>.github/workflows/ci.yml</code> on <code>global-hr-modernization</code> with <code>mvn verify</code> ; add npm <code>test</code> + <code>lint</code> jobs for <code>social-media-react</code> and <code>workbench-demo</code>	HIGH RISK	CRITICAL
Missing / incomplete <code>.gitignore</code>	Create <code>target/.gitignore</code> and <code>workbench-demo/.gitignore</code> ; extend GitHub <code>.gitignore</code> for <code>.env</code> , <code>node_modules/</code> , AI config dirs	HIGH RISK	CRITICAL
Secrets in workspace config files	Rotate exposed tokens; move values to env vars; add <code>.env.example</code> ; ensure <code>.env</code> and MCP configs are gitignored	HIGH RISK	CRITICAL
No <code>.env.example</code>	Add <code>workbench-demo/.env.example</code> documenting <code>REACT_APP_API_BASE_URL</code> ; add <code>.env.example</code> for Spring profiles using <code>\${...}</code> placeholders	HIGH RISK	HIGH
<code>ddl-auto=update</code> with no migrations	Adopt Flyway; set <code>spring.jpa.hibernate.ddl-auto=validate</code> ; create initial migration for <code>risk_register</code>	HIGH RISK	HIGH
Plaintext DB passwords in properties	Replace with env-var placeholders in <code>application-postgres.properties</code> and <code>application-oracle.properties</code>	HIGH RISK	HIGH
No containerization	Add <code>Dockerfile</code> + <code>docker-compose.yml</code> for ERM demo on port 8090; optional devcontainer for agents	HIGH RISK	MEDIUM
ESLint configured but unenforced	Add <code>lint</code> script to <code>social-media-react/package.json</code> ; enforce via CI and optional pre-commit hook	HIGH RISK	MEDIUM
Unused npm dependencies	Remove <code>dotenv</code> and <code>jwt-decode</code> from <code>social-media-react/package.json</code> or wire them into auth/config	MODERATE	LOW
Broken README doc links	Create <code>docs/START_TEST_AND_ISSUES.md</code> and <code>docs/COMPLEXITY_ISSUES.md</code> on GitHub or remove links from <code>README.md</code>	MODERATE	MEDIUM
Partial agentic module isolation	Publish a prioritized refactor queue (<code>docs/agent-tasks.md</code>) mapping Java packages and React feature folders to isolated PR-sized tasks	MODERATE	MEDIUM

7.9 Expected Outcomes

- Clean, reproducible checkout with `.gitignore` , lock files, and `.env.example` — no accidental secret commits.
- CI gate (`mvn verify` + npm test/lint) trusts agent-authored PRs before human review.
- Versioned database migrations replace `ddl-auto=update` , enabling safe schema evolution.
- Containerized local dev removes manual Maven/npm setup drift across contributors and agent sandboxes.
- Documented, enumerable work queue enables phased agentic harness adoption without cross-stack surprises.